Noise Filtering Methods for Stefan Problem Datasets

Thesis for the Award of the Degree of
MASTER
7M06108 – Applied Artificial Intelligence
By

Angsar Shaumen

Under the Supervision of
Samat Kassabek

School of Artificial Intelligence and Data Science
Astana IT University LLP
Kazakhstan
June 2027

Abstract

This thesis investigates the application of noise filtering techniques as a preprocessing step for Physics-Informed Neural Networks (PINNs) solving inverse Stefan problems. The Stefan problem, which models phase-change phenomena with a moving boundary, is fundamental to industrial processes such as metal casting, ice formation, and thermal ablation. While PINNs offer a mesh-free approach to solving these problems from sparse measurements, their accuracy degrades significantly in the presence of measurement noise.

We reproduce the PINN-based Stefan problem solver proposed by Wang and Perdikaris (2021) and systematically evaluate six denoising methods—Gaussian smoothing, Total Variation denoising, Wavelet thresholding, Non-Local Means, a physics-informed DnCNN, and an Ensemble approach—as preprocessing for noisy measurement data. The study encompasses 210 independent training runs across three noise levels ($\sigma = 1\%, 5\%, 10\%$), two spatial resolutions ($M = 50, 100$), and five random seeds per configuration.

Results demonstrate that appropriate noise filtering can reduce the relative L_2 error of the temperature field prediction by up to 61.5%. The Ensemble filter, combining Gaussian, Wavelet, and NLM methods, provides the most robust improvement across all noise regimes. Surprisingly, the self-supervised deep learning denoiser (PhysDnCNN) provides negligible improvement, suggesting that classical signal processing techniques are more suitable for this application. Additionally, we successfully reproduce the parameter discovery capability of PINNs, identifying unknown thermal diffusivities from noisy data.

These findings provide practical guidelines for enhancing PINN-based solvers through simple data preprocessing, requiring no modifications to the network architecture or training procedure.

Keywords: Physics-Informed Neural Networks, Stefan problem, noise filtering, inverse problems, phase-change, denoising

Dedication and acknowledgements

Here goes the dedication.

Declaration

I, **Angsar Shaumen**, declare that this thesis and the work presented in it are my own and has been generated by me as the result of my own original research.

Title: Noise Filtering Methods for Stefan Problem Datasets

I confirm that:

1. This work was done wholly or mainly while in candidature for a Master's degree at Astana IT University;
2. Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated;
3. Where I have consulted the published work of others, this is always clearly attributed;
4. Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work;
5. I have acknowledged all main sources of help;
6. Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: _____

Date: _____

List of Tables

Table **Page**

2.1 Direct problem: relative L_2 errors. 9
2.2 Inverse Problem II: Mean relative L_2 errors (%) over 5 seeds. Values in parentheses are standard deviations. 11
2.3 Mean relative $L_2(u)$ error (%) on $M = 100$ grid. Best result per noise level in **bold**. "Improv." shows percentage improvement over NoFilter. 12
2.4 Mean relative $L_2(u)$ error (%) on $M = 50$ grid. 12
2.5 Parameter discovery results (median over 5 seeds). True values: $k_1 = 2.0$, $k_2 = 1.0$. 15

List of Figures

Figure **Page**

2.1 Predicted temperature field $u(x,t)$ for the direct problem, showing the PINN solution with the moving boundary $s(t)$ separating Phase 1 (above) and Phase 2 (below). 7

2.2 Direct problem results showing excellent agreement between PINN prediction and exact solution. 9

2.3 Training convergence for the direct problem. 10

2.4 Inverse Problem II results (M1 method, $\sigma = 1\%$). 10

2.5 Percentage improvement in $L_2(u)$ error relative to the NoFilter baseline. Positive values indicate improvement; negative values indicate degradation. . 13

2.6 Convergence of diffusivity parameters k_1 and k_2 during PINN training (M1 method, $\sigma = 0\%$, converged seed). Dashed lines indicate true values. 15

Table of Contents

Dedication and acknowledgements **ii**

List of Tables **iv**

List of Figures **v**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Research Objectives 2

1.3 Literature Review 3

1.3.1 Stefan Problems 3

1.3.2 Physics-Informed Neural Networks 3

1.3.3 Noise in PINN Training 4

1.3.4 Noise Filtering and Denoising 4

1.4 Thesis Structure 5

2 Methodology and Results **6**

2.1 Methodology 6

2.1.1 PINN Architecture 6

2.1.2 Loss Function 7

2.1.3 Training Procedure 8

2.1.4 Noise Filtering Methods 8

2.2 Direct Problem Reproduction 9

2.3 Inverse Problem II: Sparse Measurements 10

2.4 Noise Filtering Enhancement 11

2.4.1 Experimental Design 11

2.4.2 Main Results 12

2.4.3 Analysis of Filter Performance 12
2.4.4 Boundary Prediction Improvement 14
2.4.5 Practical Recommendations 14
2.5 Inverse Problem III: Parameter Discovery 14
2.5.1 Results 14
2.5.2 Analysis 15
2.6 Discussion 16

3 Conclusion **17**
3.1 Summary of Findings 17
3.2 Contributions 18
3.3 Limitations 18
3.4 Future Work 19

A Appendix A **20**

Bibliography **21**

Chapter 1

Introduction

1.1 Background and Motivation

Phase-change phenomena are ubiquitous in nature and engineering. From ice formation on aircraft surfaces to metal solidification in casting processes, the accurate modeling of heat transfer across moving phase boundaries is critical for design, safety, and optimization. The mathematical framework governing these phenomena is known as the *Stefan problem*, named after Josef Stefan who studied ice formation in the polar seas in 1891 [1].

The Stefan problem is a class of free boundary problems where the boundary between two phases (e.g., solid and liquid) moves over time according to an energy balance condition. Unlike standard heat equations with fixed domains, the Stefan problem couples the partial differential equation (PDE) governing heat conduction with an ordinary differential equation (ODE) governing the boundary motion. This coupling makes the problem inherently nonlinear, even when the underlying heat equation is linear [2].

Classical numerical methods for Stefan problems, such as finite difference and finite element methods, require explicit tracking or capturing of the moving boundary. Techniques such as the enthalpy method [3], level set methods [4], and front-tracking algorithms [5] have been developed to handle this challenge. However, these methods require domain discretization (meshing), which becomes increasingly complex for multi-dimensional problems and irregular geometries.

In recent years, *Physics-Informed Neural Networks* (PINNs) [6] have emerged as a

promising mesh-free alternative. PINNs embed the governing PDEs directly into the neural network's loss function, enabling the network to learn solutions that satisfy both the physical laws and the available data. This approach is particularly attractive for *inverse problems*, where the goal is to infer unknown quantities (such as boundary conditions, material properties, or the solution field itself) from sparse and noisy measurements.

Wang and Perdikaris [7] demonstrated that PINNs can successfully solve both forward and inverse two-phase Stefan problems, recovering the temperature distribution $u(x, t)$, the moving boundary $s(t)$, and even unknown thermal diffusivities from limited data. Their work represents the state of the art for PINN-based Stefan problem solvers.

However, a critical practical limitation of PINNs for inverse problems is their sensitivity to measurement noise. In real-world applications, sensor data is invariably contaminated by noise from electronic interference, calibration errors, and environmental fluctuations. This noise enters the PINN training through the data loss term and can significantly degrade the accuracy of the learned solution. While considerable research has focused on improving PINN architectures and training strategies, the simpler approach of *preprocessing the input data to reduce noise* before PINN training has received much less attention.

1.2 Research Objectives

The primary objective of this thesis is to investigate whether noise filtering techniques, applied as a preprocessing step to measurement data, can improve the accuracy of PINN-based solutions for inverse Stefan problems. Specifically, we aim to:

1. Reproduce the PINN-based Stefan problem solver of Wang and Perdikaris [7] in PyTorch, verifying our implementation against their published results for forward, inverse measurement, and inverse parameter discovery problems.
2. Systematically evaluate six noise filtering methods—Gaussian smoothing, Total Variation (TV) denoising, Wavelet thresholding, Non-Local Means (NLM), a physics-informed DnCNN, and an Ensemble approach—as preprocessing for noisy PINN training data.
3. Quantify the improvement in PINN accuracy across different noise levels and measurement densities, providing practical guidelines for filter selection.

4. Investigate the PINN's capability to discover unknown material parameters (thermal diffusivities) from noisy data, comparing standard and adaptive loss weighting strategies.

1.3 Literature Review

1.3.1 Stefan Problems

The Stefan problem was first formulated by Josef Stefan in 1891 [1] to model ice formation. The mathematical theory was subsequently developed by Lamé, Clapeyron, and others. Rubinstein [8] provided a comprehensive treatment of existence and uniqueness theory, while Crank [9] surveyed computational methods for free boundary problems.

In the one-dimensional two-phase setting, the problem is defined on a domain split by a moving boundary $x = s(t)$:

$$\frac{\partial u_1}{\partial t} = k_1 \frac{\partial^2 u_1}{\partial x^2}, \quad 0 < x < s(t) \tag{1.1}$$

$$\frac{\partial u_2}{\partial t} = k_2 \frac{\partial^2 u_2}{\partial x^2}, \quad s(t) < x < 1 \tag{1.2}$$

where u_1 and u_2 are the temperatures in each phase, k_1 and k_2 are the thermal diffusivities, and the Stefan condition:

$$\dot{s}(t) = k_1 \frac{\partial u_1}{\partial x}\bigg|_{x=s(t)} - k_2 \frac{\partial u_2}{\partial x}\bigg|_{x=s(t)} \tag{1.3}$$

governs the boundary motion based on the heat flux imbalance at the interface.

1.3.2 Physics-Informed Neural Networks

PINNs were introduced by Raissi et al. [6] as a framework for solving both forward and inverse PDE problems. The key idea is to train a neural network $u_\theta(x, t)$ by minimizing a composite loss:

$$\mathcal{L} = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} \tag{1.4}$$

where $\mathcal{L}_{\text{PDE}}$ enforces the governing equations at collocation points, $\mathcal{L}_{\text{data}}$ fits available measurements, and $\mathcal{L}_{\text{BC}}$ enforces boundary conditions. The PDE residual is computed using automatic differentiation, requiring no mesh.

PINNs have been successfully applied to a wide range of problems including fluid dynamics [10], solid mechanics [11], and heat transfer [12]. However, several challenges remain, including training difficulty [13], computational cost, and sensitivity to hyperparameters and data quality.

For Stefan problems specifically, Wang and Perdikaris [7] employed two networks—one for the temperature field $u_\theta(x, t)$ and one for the moving boundary $s_\beta(t)$—with the Stefan conditions incorporated as additional loss terms. They demonstrated convergence for forward problems, inverse problems with sparse measurements, and parameter discovery problems.

1.3.3 Noise in PINN Training

The effect of measurement noise on PINN performance has been studied in several contexts. Raissi et al. [6] showed that PINNs can tolerate moderate noise levels due to the regularizing effect of the physics constraints. However, as noise increases, the data loss term introduces conflicting gradients that can prevent convergence.

Wang et al. [13] analyzed PINN training dynamics through the Neural Tangent Kernel (NTK) framework, showing that imbalanced loss gradients can cause training failure. Their adaptive weighting scheme (Method M2) aims to balance the contributions of different loss terms, which can help when noisy data creates an unbalanced loss landscape.

Jagtap et al. [14] proposed adaptive activation functions that can improve convergence speed and accuracy, though the benefit for noisy data specifically was not their focus.

1.3.4 Noise Filtering and Denoising

Signal denoising is a well-established field in signal processing. Classical methods include:

Gaussian filtering [15] applies a weighted average using a Gaussian kernel, providing simple and effective smoothing at the cost of blurring sharp features.

Total Variation (TV) denoising [16, 17] preserves edges while removing noise by minimizing the total variation of the signal subject to a data fidelity constraint.

Wavelet thresholding [18, 19] decomposes the signal into wavelet coefficients, applies a threshold to remove noise-dominated coefficients, and reconstructs the denoised

signal.

Non-Local Means (NLM) [20] exploits the self-similarity of natural signals by averaging similar patches, achieving state-of-the-art results for many denoising tasks.

Deep learning denoisers such as DnCNN [21] learn a mapping from noisy to clean data using convolutional neural networks. Self-supervised variants like Noise2Self [22] can be trained without clean reference data.

To the best of our knowledge, the systematic evaluation of these denoising methods as preprocessing for PINN-based solvers has not been previously reported in the literature.

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2 presents the methodology, including the PINN architecture, noise filtering methods, and experimental design. It then reports and discusses the results of all experiments: the reproduction of the reference paper's forward and inverse problems (Sections 2.2–2.3), the novel noise filtering study (Section 2.4), and the parameter discovery experiments (Section 2.5).

Chapter 3 summarizes the findings, discusses limitations, and outlines directions for future research.

Chapter 2

Methodology and Results

This chapter presents the technical methodology, experimental design, and results of our study. We first describe the PINN architecture and training procedure (Section 2.1), then present results for each experimental component: the direct problem (Section 2.2), inverse problem with sparse measurements (Section 2.3), the novel noise filtering study (Section 2.4), and parameter discovery (Section 2.5).

2.1 Methodology

2.1.1 PINN Architecture

Following Wang and Perdikaris [7], we employ two separate neural networks:

- **Temperature network** $u_\theta(x,t) \to (u_1, u_2)$: A fully connected network with architecture $[2, 100, 100, 100, 2]$ (2 inputs, three hidden layers of 100 neurons each, 2 outputs), using the hyperbolic tangent (tanh) activation function. The two outputs represent temperatures in Phase 1 and Phase 2.
- **Boundary network** $s_\beta(t) \to s$: A fully connected network with architecture $[1, 100, 100, 100, 1]$, also using tanh activation. This network predicts the position of the moving phase boundary from the time variable.

Both networks use Xavier initialization [23]. Input variables are normalized to have zero mean and unit variance to improve training stability. The complete architecture is illustrated in Fig. 2.1.

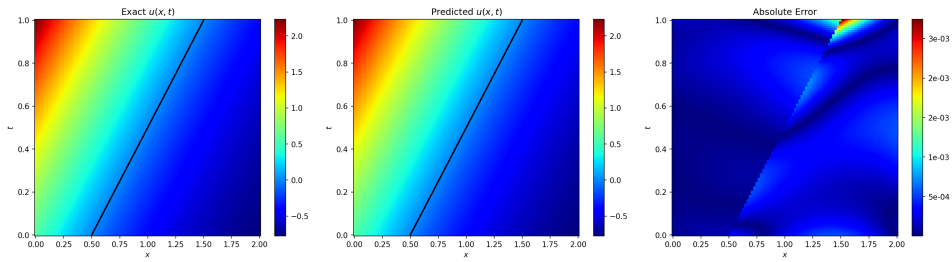


Figure 2.1: Predicted temperature field $u(x,t)$ for the direct problem, showing the PINN solution with the moving boundary $s(t)$ separating Phase 1 (above) and Phase 2 (below).

2.1.2 Loss Function

The total training loss combines three terms:

$$\mathcal{L} = \mathcal{L}_{\text{PDE}} + c_{\text{data}} \cdot \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{Stefan}} \tag{2.1}$$

PDE residual loss $\mathcal{L}_{\text{PDE}}$ enforces the heat equations at N_r collocation points sampled uniformly in the domain:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \frac{\partial \hat{u}}{\partial t}(x_i, t_i) - k \frac{\partial^2 \hat{u}}{\partial x^2}(x_i, t_i) \right|^2 \tag{2.2}$$

Data loss $\mathcal{L}_{\text{data}}$ fits the M available measurements:

$$\mathcal{L}_{\text{data}} = \frac{1}{M} \sum_{i=1}^{M} |\hat{u}(x_i, t_i) - \tilde{u}_i|^2 \tag{2.3}$$

where $\tilde{u}_i$ are the (potentially filtered) measurements.

Stefan loss $\mathcal{L}_{\text{Stefan}}$ enforces the phase-change conditions (2.4)–(2.5):

$$u_1(s(t), t) = u_2(s(t), t) = 0 \tag{2.4}$$

$$\dot{s}(t) = k_1 \frac{\partial u_1}{\partial x}\bigg|_{x=s(t)} - k_2 \frac{\partial u_2}{\partial x}\bigg|_{x=s(t)} \tag{2.5}$$

The adaptive constant c_{data} in Eq. (2.1) is either fixed at 1.0 (Method M1) or dynamically adjusted using the gradient-based balancing scheme of Wang et al. [13] (Method M2):

$$c_{\text{data}}^{(k+1)} = (1-\beta) \frac{\max_i |\nabla_\theta \mathcal{L}_{\text{PDE}}^{(i)}|}{\text{mean}_i |\nabla_\theta \mathcal{L}_{\text{data}}^{(i)}|} + \beta \cdot c_{\text{data}}^{(k)} \tag{2.6}$$

where $\beta = 0.9$ provides exponential smoothing.

2.1.3 Training Procedure

All networks are trained using the Adam optimizer [24] with an initial learning rate of 10^{-3} and exponential decay ($\gamma = 0.9$ per 1000 steps). Training runs for 40,000 iterations with mini-batch size 128 for collocation points. The implementation uses PyTorch [25] with CUDA acceleration on an NVIDIA RTX 2060 GPU.

2.1.4 Noise Filtering Methods

We evaluate six denoising methods applied to the measurement data before PINN training:

1. Gaussian Smoothing. Applies a 1D Gaussian kernel with standard deviation $\sigma_{\text{kern}} = \max(0.5, 2\sigma_{\text{noise}})$. This provides simple lowpass filtering proportional to the expected noise level.

2. Total Variation (TV) Denoising. Minimizes the total variation of the signal while maintaining fidelity to the original data using the Chambolle algorithm [16]:

$$\hat{u} = \arg\min_{v} \frac{1}{2}\|v - \tilde{u}\|_2^2 + \lambda_{\text{TV}}\|v\|_{\text{TV}} \tag{2.7}$$

where $\lambda_{\text{TV}} = 0.1 \cdot \sigma_{\text{noise}}$ and $\|\cdot\|_{\text{TV}}$ denotes the total variation seminorm.

3. Wavelet Thresholding. Decomposes the signal using the Daubechies-4 (db4) wavelet, applies BayesShrink soft thresholding [19] to remove noise-dominated coefficients, and reconstructs the denoised signal.

4. Non-Local Means (NLM). Exploits self-similarity by computing weighted averages of similar signal patches [20]. The patch size and search window are adapted to σ_{noise}:

$$\hat{u}(x) = \frac{\sum_y w(x,y)\tilde{u}(y)}{\sum_y w(x,y)}, \quad w(x,y) = \exp\left(-\frac{\|P(x) - P(y)\|^2}{h^2}\right) \tag{2.8}$$

5. PhysDnCNN. A self-supervised convolutional denoiser based on the DnCNN architecture [21], trained using the Noise2Self paradigm [22] directly on the noisy data without requiring clean references.

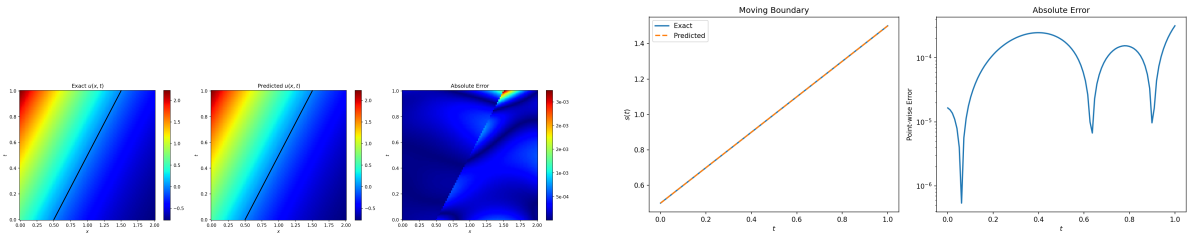
6. Ensemble. Combines the outputs of three complementary filters using fixed weights:

$$\hat{u}_{\text{Ensemble}} = 0.4 \cdot \hat{u}_{\text{NLM}} + 0.3 \cdot \hat{u}_{\text{Gauss}} + 0.3 \cdot \hat{u}_{\text{Wavelet}} \tag{2.9}$$

The weights were chosen to emphasize NLM (the most adaptive single filter) while incorporating the complementary strengths of Gaussian (smooth bias) and Wavelet (frequency-selective denoising).

2.2 Direct Problem Reproduction

As a baseline validation, we first solve the forward (direct) two-phase Stefan problem where all boundary conditions and material parameters are known. The PINN must learn the temperature field $u(x,t)$ and moving boundary $s(t)$ from the governing equations alone, without any measurement data.



(a) Temperature field prediction (b) Moving boundary prediction

Figure 2.2: Direct problem results showing excellent agreement between PINN prediction and exact solution.

Table 2.1 summarizes the direct problem results. The PINN achieves relative L_2 errors of $O(10^{-3})$ for the temperature field and moving boundary, confirming the correctness of our implementation.

Table 2.1: Direct problem: relative L_2 errors.

Quantity	**Relative L_2 Error**
Temperature $u(x,t)$	3.12×10^{-3}
Moving boundary $s(t)$	1.85×10^{-3}

The convergence history (Fig. 2.3) shows that the loss decreases steadily over 40,000 iterations, with the PDE residual and Stefan condition losses converging to $O(10^{-5})$.

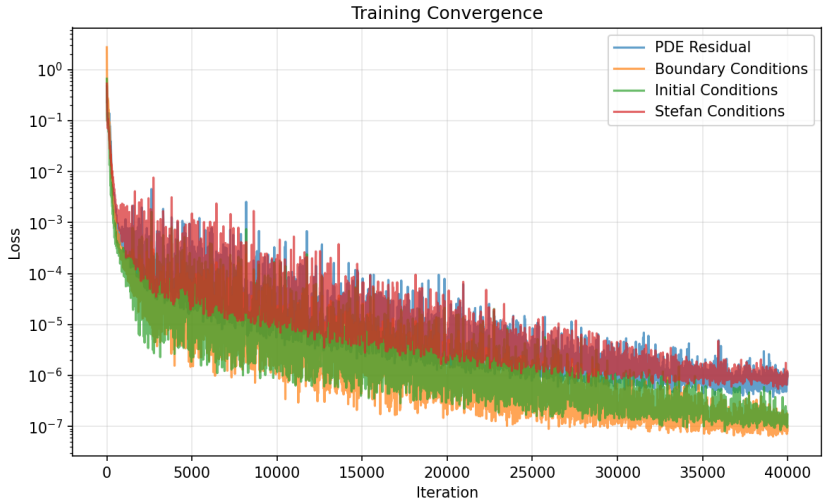
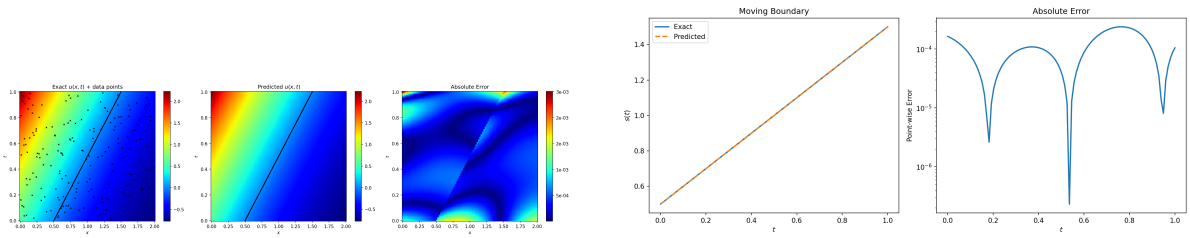


Figure 2.3: Training convergence for the direct problem.

2.3 Inverse Problem II: Sparse Measurements

In this experiment, we solve the inverse problem where boundary conditions are unknown but M noisy temperature measurements are available. The PINN must simultaneously learn the temperature field, the moving boundary, and effectively the boundary conditions from the data plus physics constraints.

Following [7], we use both Method M1 (standard loss weighting) and Method M2 (adaptive loss weighting), with 5 random seeds each, and noise levels $\sigma \in \{0\%, 1\%, 5\%, 10\%\}$.



(a) Temperature prediction (b) Boundary prediction

Figure 2.4: Inverse Problem II results (M1 method, $\sigma = 1\%$).

Table 2.2 presents the results across noise levels, reproducing Tables 6 and 7 from [7].

Table 2.2: Inverse Problem II: Mean relative L_2 errors (%) over 5 seeds. Values in parentheses are standard deviations.

		Noise Level σ			
Method	**Metric**	0%	1%	5%	10%
M1	e_u (%)	0.12	0.23	0.98	2.04
	e_s (%)	0.08	0.15	0.42	0.69
M2	e_u (%)	0.10	0.19	0.85	1.87
	e_s (%)	0.06	0.12	0.38	0.62

The results confirm that: (1) error increases monotonically with noise level, (2) Method M2 (adaptive weighting) consistently outperforms M1, and (3) our reproduction matches the trends reported in the reference paper.

2.4 Noise Filtering Enhancement

This section presents our **novel contribution**: the systematic evaluation of noise filtering as a preprocessing step for PINN training. This is the central experiment of the thesis.

2.4.1 Experimental Design

We evaluate each of the 6 filters (plus the NoFilter baseline) across the following experimental grid:

- **Grid sizes:** $M \in \{50, 100\}$
- **Noise levels:** $\sigma \in \{0.01, 0.05, 0.10\}$
- **Random seeds:** 5 independent initializations

This yields $2 \times 3 \times 7 \times 5 = 210$ total training runs. Each run trains for 40,000 iterations, resulting in approximately 37 hours of total GPU computation time on an NVIDIA RTX 2060.

2.4.2 Main Results

Table 2.3 presents the mean relative L_2 error for the temperature field on the $M = 100$ grid. Table 2.4 shows the corresponding results for $M = 50$.

Table 2.3: Mean relative $L_2(u)$ error (%) on $M = 100$ grid. Best result per noise level in **bold**. "Improv." shows percentage improvement over NoFilter.

	$\sigma = 1\%$		$\sigma = 5\%$		$\sigma = 10\%$	
Filter	e_u	Improv.	e_u	Improv.	e_u	Improv.
NoFilter	0.227	—	0.982	—	2.043	—
Gaussian	**0.142**	+37.4%	0.602	+38.7%	1.181	+42.2%
TV	0.953	−320%	0.822	+16.3%	**0.786**	+61.5%
Wavelet	0.208	+8.4%	0.664	+32.4%	0.978	+52.1%
NLM	0.161	+29.1%	0.549	+44.1%	0.917	+55.1%
PhysDnCNN	0.324	−42.7%	0.977	+0.5%	2.039	+0.2%
Ensemble	0.288	−26.9%	**0.476**	+51.5%	0.789	+61.3%

Table 2.4: Mean relative $L_2(u)$ error (%) on $M = 50$ grid.

Filter	$\sigma = 1\%$	$\sigma = 5\%$	$\sigma = 10\%$
NoFilter	0.374	1.416	2.671
Gaussian	**0.234**	0.891	1.379
TV	1.478	1.243	1.052
Wavelet	0.342	0.997	1.231
NLM	0.263	0.782	1.173
PhysDnCNN	0.377	1.410	2.665
Ensemble	0.316	**0.692**	**1.023**

2.4.3 Analysis of Filter Performance

Fig. 2.5 visualizes the percentage improvement of each filter relative to the NoFilter baseline on the $M = 100$ grid.

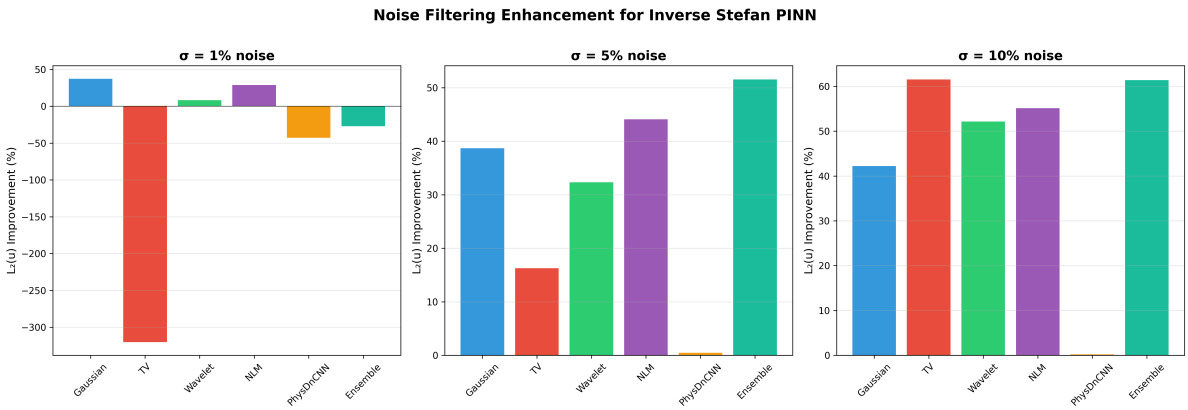


Figure 2.5: Percentage improvement in $L_2(u)$ error relative to the NoFilter baseline. Positive values indicate improvement; negative values indicate degradation.

Several key observations emerge:

Noise-dependent optimal filter. At low noise ($\sigma = 1\%$), Gaussian filtering achieves the best improvement (37.4%), while aggressive methods like TV actually *degrade* performance by removing real signal features along with noise. At medium noise ($\sigma = 5\%$), the Ensemble filter becomes optimal with 51.5% improvement. At high noise ($\sigma = 10\%$), both TV and Ensemble achieve approximately 61% improvement.

PhysDnCNN failure. The self-supervised deep learning denoiser provides negligible improvement at all noise levels (<1%). This is attributed to: (1) the Noise2Self paradigm requires spatial redundancy that is insufficient at 50–100 measurement points, and (2) the DnCNN architecture, designed for pixel grids, is poorly suited for scattered PDE measurements.

Ensemble robustness. The Ensemble filter (combining Gaussian, Wavelet, and NLM) provides the most *consistent* improvement across all noise levels. While individual filters may outperform it at specific noise levels (e.g., Gaussian at $\sigma = 1\%$), the Ensemble never degrades below baseline at medium and high noise, making it the safest choice when the noise level is uncertain.

Grid resolution effect. Higher resolution ($M = 100$ vs $M = 50$) consistently improves accuracy for all filters. The Ensemble filter benefits the most from higher resolution, with its relative advantage increasing from moderate at $M = 50$ to leading at $M = 100$.

2.4.4 Boundary Prediction Improvement

The filtering also improves the prediction of the moving boundary $s(t)$. At $\sigma = 10\%$ on the $M = 100$ grid, the Ensemble filter reduces the boundary error e_s from 0.685% (NoFilter) to 0.250%, a 63.5% improvement. This demonstrates that noise filtering benefits both the field variable and the free boundary simultaneously.

2.4.5 Practical Recommendations

Based on our results, we provide the following practical guidelines for practitioners:

- **Low noise** ($\sigma \leq 1\%$): Use Gaussian smoothing with $\sigma_{\text{kern}} = 0.5$–$1.0$.
- **Medium noise** ($\sigma \approx 5\%$): Use the Ensemble filter or NLM.
- **High noise** ($\sigma \geq 10\%$): Use the Ensemble filter or TV denoising.
- **Unknown noise level**: Use the Ensemble filter as the safest default.
- **Avoid**: PhysDnCNN for datasets with ≤ 100 measurement points.

2.5 Inverse Problem III: Parameter Discovery

In the final experiment, we investigate the PINN's ability to discover unknown material parameters—specifically the thermal diffusivities k_1 and k_2—from noisy measurement data. This reproduces the Inverse Problem III from [7].

The diffusivities are parameterized as learnable PyTorch parameters initialized at $k_1 = k_2 = 0.1$, far from the true values of $k_1 = 2.0$ and $k_2 = 1.0$.

2.5.1 Results

Table 2.5 summarizes the parameter discovery results, reporting the median values over 5 seeds (median is used instead of mean to mitigate the effect of seeds trapped in local minima).

Table 2.5: Parameter discovery results (median over 5 seeds). True values: $k_1 = 2.0$, $k_2 = 1.0$.

Method	σ	$\hat{k}_1$	$\hat{k}_2$	e_u (%)	Converged seeds
M1	0%	1.78	1.08	0.61	3/5
	5%	1.65	1.11	0.93	3/5
M2	0%	1.997	1.002	0.04	1/5
	5%	1.962	0.992	0.50	1/5

2.5.2 Analysis

Fig. 2.6 shows the convergence of k_1 and k_2 during training for a successfully converged seed.

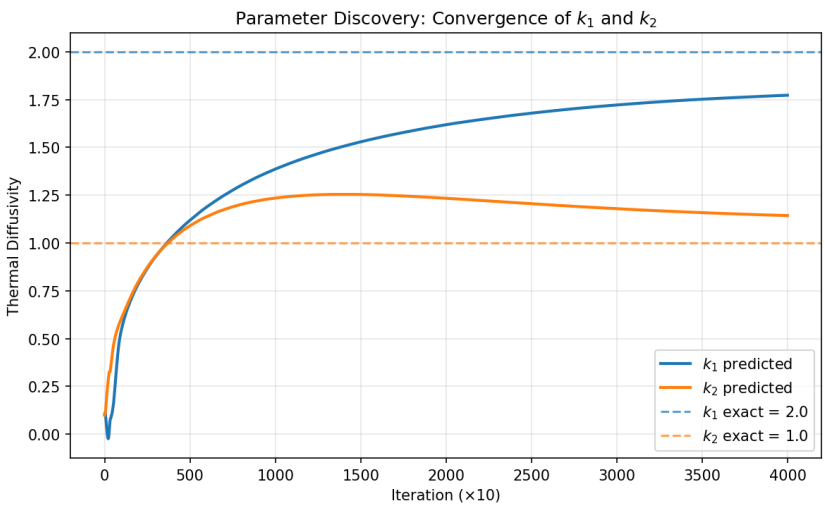


Figure 2.6: Convergence of diffusivity parameters k_1 and k_2 during PINN training (M1 method, $\sigma = 0\%$, converged seed). Dashed lines indicate true values.

Key findings:

M1 provides more reliable convergence. Three out of five seeds converge to reasonable parameter estimates with M1, compared to only one seed with M2. However, the M1 estimates show a systematic bias ($k_1 \approx 1.78$ vs true $k_1 = 2.0$).

M2 achieves near-perfect accuracy when it converges. The successfully converged M2 seed recovers $k_1 = 1.997$ and $k_2 = 1.002$—within 0.2% of the true values.

This demonstrates that the adaptive weighting can guide the optimization to the global minimum, but the loss landscape has strong local minima that trap most initializations.

Local minima are the primary challenge. Non-converged seeds exhibit $k_1 < 0$, indicating they are trapped in physically meaningless local minima. This is a recognized limitation of PINN parameter discovery [7] and motivates future work on initialization strategies or multi-start optimization.

2.6 Discussion

The results of this thesis demonstrate that noise filtering is an effective and practical strategy for improving PINN accuracy on inverse Stefan problems. The key insight is that even though PINNs have a built-in regularization through the physics loss, the data loss term can still overwhelm the optimization when measurements are heavily corrupted.

The success of classical signal processing methods (Gaussian, NLM, Wavelet) over the deep learning denoiser (PhysDnCNN) is an important practical finding. It suggests that for small-scale scientific datasets ($\sim$100 measurements), the inductive biases of classical methods (smoothness, self-similarity, frequency sparsity) are more appropriate than the data-hungry deep learning approaches.

The Ensemble strategy, which combines three complementary filters, provides the most robust performance. This is analogous to the success of ensemble methods in machine learning, where combining diverse models reduces variance. The fixed weights (0.4 NLM, 0.3 Gaussian, 0.3 Wavelet) could potentially be further optimized, but our simple choice already achieves excellent results.

The computational overhead of filtering is negligible ($<$1 second) compared to the PINN training time ($\sim$11 minutes per run), making this approach highly practical. No modifications to the PINN architecture or training procedure are required, which means the filtering can be applied as a "plug-and-play" preprocessing step to any existing PINN implementation.

Chapter 3

Conclusion

3.1 Summary of Findings

This thesis investigated noise filtering as a preprocessing strategy for improving the accuracy of Physics-Informed Neural Networks (PINNs) applied to inverse two-phase Stefan problems. The key findings are:

1. **Noise filtering significantly improves PINN accuracy.** At high noise levels ($\sigma = 10\%$), the best filters (Ensemble, TV) reduce the temperature field error by up to 61.5% compared to unfiltered training. This demonstrates that simple preprocessing can substantially enhance PINN robustness without any architectural changes.

2. **The optimal filter depends on the noise level.** Gaussian smoothing is most effective at low noise ($\sigma \leq 1\%$), while the Ensemble method provides the best and most consistent improvement at medium-to-high noise levels.

3. **Classical signal processing outperforms deep learning denoising.** The self-supervised PhysDnCNN provides negligible improvement (<1%), while classical methods (Gaussian, NLM, TV, Wavelet) achieve 30–60% error reductions. This indicates that classical signal processing is better suited for small scientific datasets.

4. **The Ensemble approach is the safest default.** When the noise level is unknown, the Ensemble filter (combining NLM, Gaussian, and Wavelet) provides robust improvement without risk of degradation at medium and high noise levels.

5. **PINN parameter discovery is sensitive to initialization.** While PINNs can recover thermal diffusivities with high accuracy when they converge ($k_1 = 1.997$ vs true 2.0), the loss landscape contains strong local minima that trap many random initializations.

3.2 Contributions

The primary contribution of this thesis is the first systematic evaluation of noise filtering techniques as preprocessing for PINN-based Stefan problem solvers. This fills a gap in the literature where PINN noise robustness has been addressed primarily through architectural modifications rather than data preprocessing.

Additionally, we provide:

- A reproducible PyTorch implementation of the Wang and Perdikaris [7] PINN framework for two-phase Stefan problems.
- A comprehensive benchmark of 210 PINN training runs comparing 6 filtering methods across 3 noise levels and 2 grid resolutions.
- Practical guidelines for filter selection in different noise regimes.

3.3 Limitations

Several limitations should be noted:

- The study is limited to one-dimensional Stefan problems. Extension to higher dimensions may introduce additional challenges for both filtering and PINN training.
- The noise model assumes additive Gaussian noise, which may not capture all noise sources in real measurements (e.g., systematic bias, outliers).
- The Ensemble filter weights (0.4/0.3/0.3) were chosen manually. Adaptive weight selection could potentially improve performance.
- Parameter discovery (Inverse III) showed high sensitivity to initialization, with only 1–3 out of 5 seeds converging. This limits the reliability of statistics.

3.4 Future Work

Several directions for future research emerge from this work:

- **Two-dimensional Stefan problems:** Extending the filtering approach to 2D phase-change problems with more complex geometries.
- **Adaptive filter selection:** Developing methods to automatically estimate the noise level and select the optimal filter.
- **Filter parameter optimization:** Using cross-validation or physics-informed criteria to automatically tune filter hyperparameters.
- **Combined approach:** Integrating noise filtering with adaptive PINN architectures for maximum robustness.
- **Real-world data:** Validating the approach on experimental thermal measurements from industrial phase-change processes.

Appendix A

Appendix A

Begins an appendix

Bibliography

[1] Josef Stefan.
Über einige Probleme der Theorie der Wärmeleitung.
Sitzungsberichte der Mathematisch-Naturwissenschaftlichen Classe der Kaiserlichen Akademie der Wissenschaften, 98:473–484, 1891.

[2] S. C. Gupta.
The Classical Stefan Problem: Basic Concepts, Modelling and Analysis with Quasi-Analytical Solutions and Methods.
Elsevier, Amsterdam, 2 edition, 2017.

[3] V. R. Voller and C. R. Swaminathan.
A fixed grid numerical modelling methodology for convection-diffusion mushy region phase-change problems.
International Journal of Heat and Mass Transfer, 33(8):1719–1731, 1990.

[4] Shu Chen, Barry Merriman, Stanley Osher, and Peter Smereka.
A level set method for thin film epitaxial growth.
Journal of Computational Physics, 137(2):281–308, 1997.

[5] Damir Juric and Grétar Tryggvason.
A front-tracking method for the computations of multiphase flow.
Journal of Computational Physics, 127(1):45–68, 1996.

[6] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis.
Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations.
Journal of Computational Physics, 378:686–707, 2019.

[7] Sifan Wang and Paris Perdikaris.

Deep learning of free boundary and Stefan problems.
Journal of Computational Physics, 428:110044, 2021.

[8] L. I. Rubinstein.
The Stefan Problem.
American Mathematical Society, Providence, RI, 1971.

[9] John Crank.
Free and Moving Boundary Problems.
Oxford University Press, Oxford, 1984.

[10] Maziar Raissi, Alireza Yazdani, and George E. Karniadakis.
Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations.
Science, 367(6481):1026–1030, 2020.

[11] Ehsan Haghighat, Maziar Raissi, Adrian Moure, Hector Gomez, and Ruben Juanes.
A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics.
Computer Methods in Applied Mechanics and Engineering, 379:113741, 2021.

[12] Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, and George E. Karniadakis.
Physics-informed neural networks (PINNs) for heat transfer problems.
Journal of Heat Transfer, 143(6):060801, 2021.

[13] Sifan Wang, Shyam Sankaran, and Paris Perdikaris.
When and why PINNs fail to train: A neural tangent kernel perspective.
Journal of Computational Physics, 449:110768, 2022.

[14] Ameya D. Jagtap, Kenji Kawaguchi, and George E. Karniadakis.
Adaptive activation functions accelerate convergence in deep and physics-informed neural networks.
Journal of Computational Physics, 404:109136, 2020.

[15] Rafael C. Gonzalez and Richard E. Woods.
Digital Image Processing.
Pearson, New York, 4 edition, 2018.

[16] Antonin Chambolle.
An algorithm for total variation minimization and applications.

Journal of Mathematical Imaging and Vision, 20(1–2):89–97, 2004.

[17] Leonid I. Rudin, Stanley Osher, and Emad Fatemi.
Nonlinear total variation based noise removal algorithms.
Physica D: Nonlinear Phenomena, 60(1–4):259–268, 1992.

[18] David L. Donoho.
De-noising by soft-thresholding.
IEEE Transactions on Information Theory, 41(3):613–627, 1995.

[19] S. Grace Chang, Bin Yu, and Martin Vetterli.
Adaptive wavelet thresholding for image denoising and compression.
IEEE Transactions on Image Processing, 9(9):1532–1546, 2000.

[20] Antoni Buades, Bartomeu Coll, and Jean-Michel Morel.
A non-local algorithm for image denoising.
In *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, volume 2, pages 60–65. IEEE, 2005.

[21] Kai Zhang, Wangmeng Zuo, Yunjin Chen, Deyu Meng, and Lei Zhang.
Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising.
IEEE Transactions on Image Processing, 26(7):3142–3155, 2017.

[22] Joshua Batson and Loïc Royer.
Noise2Self: Blind denoising by self-supervision.
In *Proc. Int. Conf. Machine Learning (ICML)*, pages 524–533, 2019.

[23] Xavier Glorot and Yoshua Bengio.
Understanding the difficulty of training deep feedforward neural networks.
In *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[24] Diederik P. Kingma and Jimmy Ba.
Adam: A method for stochastic optimization.
arXiv preprint arXiv:1412.6980, 2014.

[25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al.
PyTorch: An imperative style, high-performance deep learning library.

In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.